

Algorithmie – Récursivité et Problème Classique

Table des matières

1 - Consolidation des fonctions	2
Rappel rapide : paramètres et retour	2
Importance pour la récursivité	2
2 - Introduction à la récursivité	3
Définition : une fonction qui s'appelle elle-même	3
Analogie : poupées russes, miroirs face à face	3
Éléments essentiels d'une fonction récursive	4
Danger : boucle infinie si pas de cas de base	5
3 - Exemples progressifs	6
Exemple 1 : Compte à rebours récursif (simple)	6
Exemple 2 : Factorielle ($n! = n \times (n-1)!$)	7
Exemple 3 : Suite de Fibonacci ($Fib(n) = Fib(n-1) + Fib(n-2)$)	10
4 - Problème classique : Tour de Hanoï	12
Présentation du problème (3 tours, disques)	12
Règles du jeu	14
Solution récursive (explication conceptuelle)	14
Visualisation simplifiée (2-3 disques)	16
Nombre de mouvements : $2^n - 1$	18
5 - Exercices pratiques	19
Exercice 1 : Somme récursive	19
Exercice 2 : Puissance récursive	19
Exercice 3 : Comprendre la Tour de Hanoï	20

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

1- Consolidation des fonctions

Rappel rapide : paramètres et retour

Avant d'aborder la récursivité, rappelons les concepts essentiels des fonctions.

Une fonction peut :

- Recevoir des **paramètres** (données en entrée)
- Effectuer des **traitements**
- **Retourner** un résultat

Exemple simple :

```
FONCTION carre(nombre)
    resultat ← nombre × nombre
    RETOURNER resultat
FIN FONCTION

x ← carre(5) → x = 25
```

Importance pour la récursivité

La récursivité repose sur le principe qu'**une fonction peut s'appeler elle-même**.

Pour comprendre la récursivité, il faut maîtriser :

- Les paramètres
- La valeur de retour
- Le déroulement d'un appel de fonction

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

2- Introduction à la récursivité

Définition : une fonction qui s'appelle elle-même

Une **fonction récursive** est une fonction qui, dans son corps, fait **appel à elle-même**.

Structure de base :

```
FONCTION fonctionRecursive(parametre)
    SI condition d'arrêt
        RETOURNER valeur de base
    SINON
        RETOURNER fonctionRecursive(parametre modifié)
    FIN SI
FIN FONCTION
```

Analogie : poupées russes, miroirs face à face

Poupées russes (Matriochkas) :

- Une grande poupée contient une plus petite
- Cette petite contient une encore plus petite
- Jusqu'à la plus petite qui ne contient rien
- C'est exactement comme la récursivité !

Miroirs face à face :

- Un miroir reflète l'autre
- Qui reflète le premier
- Qui reflète le deuxième
- À l'infini... (danger !)

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Éléments essentiels d'une fonction récursive

1. Le cas de base (condition d'arrêt)

- C'est la condition qui **arrête** la récursivité
- Sans cas de base → boucle infinie !
- C'est le cas le plus simple à résoudre

2. Le cas récursif (appel à soi-même)

- La fonction s'appelle elle-même
- Avec des **paramètres modifiés** (se rapprocher du cas de base)
- Sinon → boucle infinie !

Schéma obligatoire :

```
FONCTION recursive(n)
    SI cas de base
        RETOURNER solution simple
    SINON
        RETOURNER recursive(n modifié) + traitement
    FIN SI
FIN FONCTION
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Danger : boucle infinie si pas de cas de base

Exemple DANGEREUX :

```
FONCTION mauvaise(n)
    RETOURNER mauvaise(n) // Pas de cas de base !
FIN FONCTION
```

Résultat : la fonction s'appelle sans fin → plantage !

Exemple CORRECT :

```
FONCTION bonne(n)
    SI n = 0 // Cas de base
        RETOURNER 1
    SINON
        RETOURNER bonne(n - 1) // Se rapproche du cas de base
    FIN SI
FIN FONCTION
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

3- Exemples progressifs

Exemple 1 : Compte à rebours récursif (simple)

Objectif : Afficher les nombres de N à 0

Version itérative (avec boucle) :

```
POUR i de N à 0 (décroissant)
    AFFICHER i
FIN POUR
```

Version récursive :

```
FONCTION compteAREbours(n)
    SI n < 0 // Cas de base
        // Ne rien faire, arrêt
    SINON
        AFFICHER n
        compteAREbours(n - 1) // Appel récursif
    FIN SI
FIN FONCTION
```

Utilisation :

```
compteAREbours(5)
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Traçage de l'exécution :

```
compteAREbours(5) → affiche 5 → appelle compteAREbours(4)
compteAREbours(4) → affiche 4 → appelle compteAREbours(3)
compteAREbours(3) → affiche 3 → appelle compteAREbours(2)
compteAREbours(2) → affiche 2 → appelle compteAREbours(1)
compteAREbours(1) → affiche 1 → appelle compteAREbours(0)
compteAREbours(0) → affiche 0 → appelle compteAREbours(-1)
compteAREbours(-1) → cas de base, arrêt
```

Résultat : `5 4 3 2 1 0`

Exemple 2 : Factorielle ($n! = n \times (n-1)!$)

Définition mathématique :

$$n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$$
$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Cas particulier : $0! = 1$

Définition récursive :

$$n! = n \times (n-1)!$$
$$0! = 1 \quad (\text{cas de base})$$

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Algorithme récursif :

```
FONCTION factorielle(n)
  SI n = 0 // Cas de base
    RETOURNER 1
  SINON
    RETOURNER n × factorielle(n - 1) // Cas récursif
  FIN SI
FIN FONCTION
```

Traçage de l'exécution pour factorielle(4) :

```
factorielle(4)
  ↓ appelle factorielle(3)
factorielle(3)
  ↓ appelle factorielle(2)
factorielle(2)
  ↓ appelle factorielle(1)
factorielle(1)
  ↓ appelle factorielle(0)
factorielle(0) → retourne 1
  ← retourne 1 × 1 = 1
  ← retourne 2 × 1 = 2
  ← retourne 3 × 2 = 6
  ← retourne 4 × 6 = 24
```

Résultat : `factorielle(4) = 24`

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☑ Jérôme CHRETIENNE
☑ Sophie POULAKOS
☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Explication détaillée :

1. ``factorielle(4)`` appelle ``factorielle(3)``
2. ``factorielle(3)`` appelle ``factorielle(2)``
3. ``factorielle(2)`` appelle ``factorielle(1)``
4. ``factorielle(1)`` appelle ``factorielle(0)``
5. ``factorielle(0)`` retourne 1 (cas de base)
6. ``factorielle(1)`` retourne $1 \times 1 = 1$
7. ``factorielle(2)`` retourne $2 \times 1 = 2$
8. ``factorielle(3)`` retourne $3 \times 2 = 6$
9. ``factorielle(4)`` retourne $4 \times 6 = 24$

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Exemple 3 : Suite de Fibonacci ($\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$)

Définition de la suite de Fibonacci :

```
Fib(0) = 0
Fib(1) = 1
Fib(n) = Fib(n-1) + Fib(n-2)

Suite : 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...
```

Chaque terme = somme des deux précédents

Algorithme récursif :

```
FONCTION fibonacci(n)
  SI n = 0 // Premier cas de base
    RETOURNER 0
  SINON SI n = 1 // Deuxième cas de base
    RETOURNER 1
  SINON
    RETOURNER fibonacci(n - 1) + fibonacci(n - 2) // Cas récursif
  FIN SI
FIN FONCTION
```

Attention : Ici on a **deux cas de base** (n=0 et n=1) !

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☑ Jérôme CHRETIENNE
☑ Sophie POULAKOS
☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

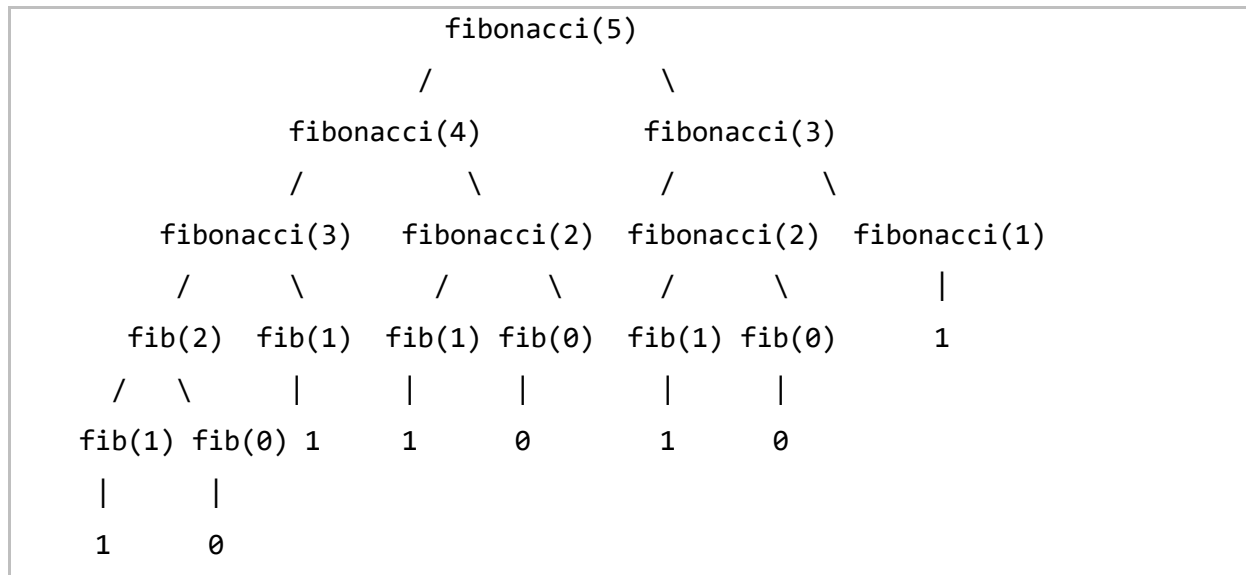
xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Traçage de l'exécution pour fibonacci(5) :



Calcul :

```

fibonacci(5) = fibonacci(4) + fibonacci(3)
              = (fibonacci(3) + fibonacci(2)) + (fibonacci(2) +
fibonacci(1))
              = ((2 + 1) + (1 + 1)) + ((1 + 1) + 1)
              = (3 + 2) + (2 + 1)
              = 5 + 3
              = 8
  
```

Résultat : `fibonacci(5) = 8`

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☑ Jérôme CHRETIENNE
 ☑ Sophie POULAKOS
 ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Note importante : Fibonacci récursif est **inefficace** car il recalcule les mêmes valeurs plusieurs fois. C'est un bon exemple pédagogique mais pas optimal en pratique.

4- Problème classique : Tour de Hanoï

Présentation du problème (3 tours, disques)

La **Tour de Hanoï** est un casse-tête mathématique classique.

Matériel :

- 3 tours (A, B, C)
- N disques de tailles différentes

Configuration initiale :

- Tous les disques sont sur la tour A
- Rangés du plus grand (en bas) au plus petit (en haut)

Objectif :

- Déplacer tous les disques de la tour A vers la tour C
- En utilisant la tour B comme intermédiaire

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Visualisation avec 3 disques :

État initial :

[1]		
[222]		
[33333]		
A	B	C

État final :

		[1]
		[222]
		[33333]
A	B	C

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Règles du jeu

3 règles obligatoires :

1. Un seul disque déplacé à la fois

- On ne peut déplacer qu'un disque par mouvement

2. Toujours prendre le disque du dessus

- On ne peut prendre qu'un disque qui est sur le dessus d'une tour

3. Un grand disque ne peut jamais être sur un petit

- On ne peut poser un disque que sur un disque plus grand (ou sur une tour vide)

Solution récursive (explication conceptuelle)

Idée clé : Pour déplacer N disques de A vers C :

1. Déplacer les N-1 disques du dessus de A vers B (en utilisant C comme intermédiaire)
2. Déplacer le plus grand disque de A vers C
3. Déplacer les N-1 disques de B vers C (en utilisant A comme intermédiaire)

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Algorithme récursif :

```
FONCTION hanoi(n, depart, arrivee, intermediaire)
  SI n = 1 // Cas de base : un seul disque
    AFFICHER "Déplacer disque 1 de " + depart + " vers " + arrivee
  SINON
    // Étape 1 : déplacer n-1 disques vers l'intermédiaire
    hanoi(n - 1, depart, intermediaire, arrivee)

    // Étape 2 : déplacer le plus grand disque
    AFFICHER "Déplacer disque " + n + " de " + depart + " vers " +
arrivee

    // Étape 3 : déplacer n-1 disques de l'intermédiaire vers
l'arrivée
    hanoi(n - 1, intermediaire, arrivee, depart)
  FIN SI
FIN FONCTION
```

Utilisation :

```
hanoi(3, "A", "C", "B")
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Visualisation simplifiée (2-3 disques)

Avec 2 disques :

```
hanoi(2, A, C, B)
├─ hanoi(1, A, B, C)    → "Déplacer disque 1 de A vers B"
├─ "Déplacer disque 2 de A vers C"
└─ hanoi(1, B, C, A)    → "Déplacer disque 1 de B vers C"
```

Mouvements :

1. Déplacer disque 1 de A vers B
2. Déplacer disque 2 de A vers C
3. Déplacer disque 1 de B vers C

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Avec 3 disques :

```

hanoi(3, A, C, B)
├─ hanoi(2, A, B, C)
|   ├─ hanoi(1, A, C, B)    → "1: A → C"
|   ├─ "2: A → B"
|   └─ hanoi(1, C, B, A)    → "1: C → B"
├─ "3: A → C"
└─ hanoi(2, B, C, A)
    ├─ hanoi(1, B, A, C)    → "1: B → A"
    ├─ "2: B → C"
    └─ hanoi(1, A, C, B)    → "1: A → C"
  
```

Mouvements :

1. Déplacer disque 1 de A vers C
2. Déplacer disque 2 de A vers B
3. Déplacer disque 1 de C vers B
4. Déplacer disque 3 de A vers C
5. Déplacer disque 1 de B vers A
6. Déplacer disque 2 de B vers C
7. Déplacer disque 1 de A vers C

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Nombre de mouvements : $2^n - 1$

Formule mathématique :

Pour N disques, le nombre minimum de mouvements est : $2^n - 1$

Exemples :

- 1 disque : $2^1 - 1 = 1$ mouvement
- 2 disques : $2^2 - 1 = 3$ mouvements
- 3 disques : $2^3 - 1 = 7$ mouvements
- 4 disques : $2^4 - 1 = 15$ mouvements
- 5 disques : $2^5 - 1 = 31$ mouvements
- 10 disques : $2^{10} - 1 = 1023$ mouvements
- 64 disques : $2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615$ mouvements

Légende : Avec 64 disques à 1 mouvement par seconde, il faudrait 585 milliards d'années !

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

5- Exercices pratiques

Exercice 1 : Somme récursive

Écrire une fonction récursive `somme(n)` qui calcule la somme de tous les nombres de 1 à n.

Exemple : `somme(5) = 1 + 2 + 3 + 4 + 5 = 15`

Questions :

1. Identifier le cas de base
2. Identifier le cas récursif
3. Écrire l'algorithme complet
4. Tracer l'exécution pour n=4

Notions travaillées :

- Structure récursive
- Cas de base et cas récursif
- Traçage

Exercice 2 : Puissance récursive

Écrire une fonction récursive `puissance(x, n)` qui calcule x^n .

Exemple : `puissance(2, 4) = 2^4 = 2 × 2 × 2 × 2 = 16`

Questions :

1. Quel est le cas de base ? (rappel : $x^0 = 1$)
2. Quelle est la relation récursive ? ($x^n = x \times x^{(n-1)}$)
3. Écrire l'algorithme complet
4. Tracer l'exécution pour puissance(2, 4)

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Récursivité et Problème Classique

Notions travaillées :

- Fonction avec 2 paramètres
- Cas de base mathématique
- Réduction du problème

Exercice 3 : Comprendre la Tour de Hanoï

1. Résoudre manuellement la Tour de Hanoï avec **3 disques**
2. Lister tous les mouvements nécessaires
3. Vérifier qu'on a bien $2^3 - 1 = 7$ mouvements
4. Écrire l'algorithme récursif complet
5. Compter le nombre de mouvements nécessaires pour 5 disques (sans les lister)

Notions travaillées :

- Compréhension de la récursivité
- Problème classique
- Formule mathématique

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.